Phase 1 Proposal: Course-Advising Assistant for JUST CIS Students

Building LLM Systems: From Models to Production · Team PlanPath

1. Team and roles

Member	Phase 1 role	Responsible for
Gina Maayah	Engineering lead + Evaluation lead	Repository, architecture, free-compute check; seed test set and scoring rules
Tala alakash	Data lead	Collecting the study plan data, data card
Saja Al_Dagamseh	Product lead	User research, problem section

2. User and problem

User: Computer Information Systems (CIS) students at Jordan University of Science and Technology (JUST), mainly those in their 2nd and 3rd years, are preparing to select courses for the upcoming semester.

What they struggle with: Before each registration period, students must determine which courses they are eligible to take (based on prerequisites), identify the requirement group for each course, and calculate how many credit hours remain in each group. The official study plan contains this information, but it is presented as a lengthy table divided into several requirement groups, with prerequisites listed for each course. To answer a question like "Can I take course X if I have passed A and B?" students have to manually search through the table.

Current method: browsing the department website for the study plan, googling, asking peers or student WhatsApp groups (where answers come from other students and are often unverified), or waiting for an appointment with their academic advisor.

Evidence: our team members are JUST students who plan each semester's registration from this study plan and have needed precisely this information (prerequisites, credit hours per requirement group, required vs. elective). The test set is built from these question types, with every answer taken from the plan itself.

3. What the system will do

The system responds to students' questions about the official JUST CIS study plan. The answers are in English or Arabic. Each answer includes the course code and the section of the study plan in which it appears, so the student can check it. In cases not covered by the study plan (who is teaching the course, which sections are open this semester, exceptions to policies), the system says it does not know and suggests the student seek help from their academic advisor or the department.

Sample dialogues (expected answers taken from the official study plan):

User input	Expected answer
How many University Elective hours do I need?	9 credit hours (University Elective Requirements, CIS study plan no. 766).
I only took HSS112SE. Can I register for CIs381 next semester?	"No. CIs381 Mobile Application Development also requires CIs341 (Department Compulsory Requirements, CIS study plan no. 766)."
Who is teaching CIs381 this semester?	"I don't know. Teaching assignments are not listed in the study plan; please refer to the department's semester schedule."

4. Data

All data is public, published by JUST, and already available (see details in the data card).

Source	Content	Format
JUST B.Sc. CIS study plan, plan no. 766	All courses with code, course name, credit hours, prerequisite, and teaching method, grouped into requirement sections (University Compulsory 16 h, University Elective 9 h, Faculty Compulsory 24 h, Department Compulsory 71 h, Department Elective 12 h).	HTML table
Course syllabi and descriptions, linked from each row of the plan	Course topics, objectives, assessment	Web pages / PDF
CIS department Study Plans page	Links to the study plans for all department programs.	HTML

Size: 256 table rows forming 115 courses in 5 requirement sections (132 credit hours), plus 768 linked syllabus, course description and course evaluation documents. **Processing:** the plan table was converted into a structured CSV (code, name, hours, prerequisites, equivalents, section). Automatic checks passed, and 10 random rows matched the official page when checked by hand. **Sensitive content:** none; the data contains no personal information.

5. Planned architecture

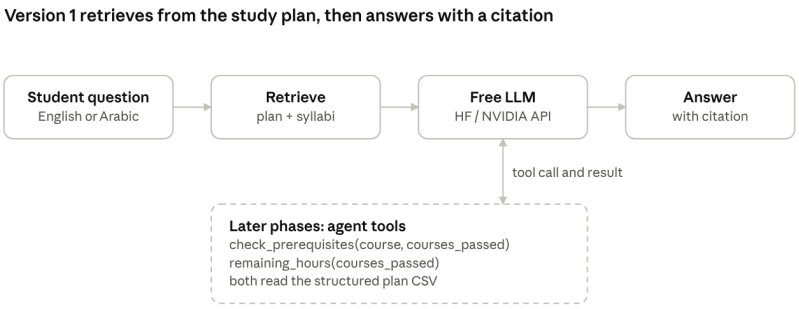


Figure 1: Planned architecture (version 1, with agent tools added in later phases).

Version 1 (Phase 2): retrieval-augmented generation (RAG). The study plan and syllabi are divided into chunks (one chunk for each course row, plus syllabus sections), embedded using a small open embedding model, and stored in a local vector index on Colab. The top relevant chunks are sent to a free LLM via the Hugging Face or NVIDIA API, which generates answers and cites the course code and plan section.

Second technique (later phases): an agent with two deterministic Python tools over the structured plan CSV, so that eligibility and credit-hour questions are computed, not guessed. We retain the agent only if it outperforms plain RAG on our test set.

6. How we will measure success

We evaluate on our seed test set: 30 questions now, growing to at least 50 in Phase 2. Every expected answer is written from the official plan before any model is tried. About half the questions are easy (one fact), a third medium (a prerequisite chain or a syllabus topic), and the rest hard, including at least 3 that the system must refuse.

An answer is correct if: its key fact (course code, credit hours, Yes/No eligibility, or missing prerequisite) matches the expected answer exactly; for refusal questions, the system says it does not know and does not invent an answer.

Numbers we will report in every phase:

- Accuracy: the proportion of test set questions answered correctly.
- Citation accuracy: the proportion of answers that cite the correct course and plan section.
- Refusal accuracy: the proportion of questions that should be refused and are correctly refused.
- Accuracy broken down by difficulty level (easy, medium, hard), allowing us to identify where each additional technique helps.

7. Risks and backup plan

Risk	Type	Backup plan
The plan page changes, or prerequisites are misread when parsed	Data	Keep a saved copy of the page; after every new download, rerun the automatic checks and re-check 10 rows by hand
The data is small, so pasting the whole plan into the prompt may score as well as retrieval	Scope	Report the paste-everything version as our baseline; add the syllabi to the retrieval corpus; keep a technique only if it beats the baseline on our test set
The LLM makes mistakes on prerequisite chains or credit-hour counts	Model	Move these to deterministic Python tools called by the agent
Free models perform poorly on Arabic questions	Model	Measure it in Phase 2; if it is poor, support English first and document Arabic results separately
Free API rate limits or Colab session limits	Compute	Small corpus and small embedding model; cache embeddings; fall back to a smaller open model on Colab
Study plans differ by entry year	Scope	Fix one plan (no. 766) and state it in every answer